

The C++ Stack

LIFO Data Structure Guide

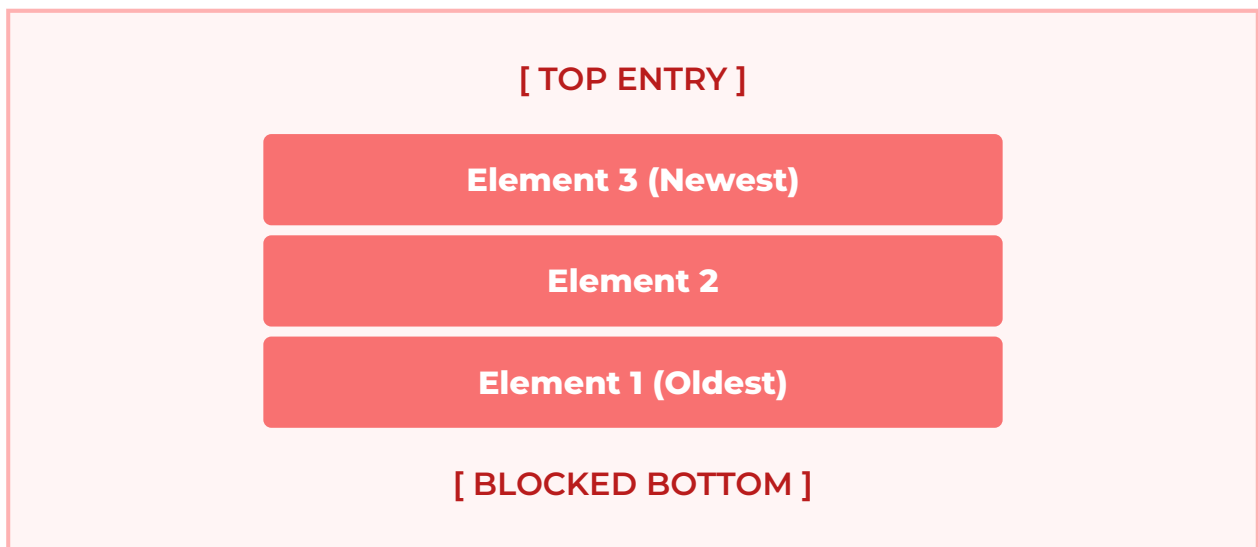
A Comprehensive 14-Page Technical Manual

Last In, First Out Philosophy | Applications | Implementation

1. Introduction to the Stack

A **Stack** is a linear container adapter that provides a specific, restricted way to store and retrieve data. It follows the **LIFO (Last In, First Out)** principle.

In a stack, you can only interact with the "top" of the data structure. You cannot reach into the middle or the bottom without removing everything above it first.



Real-Life Analogy

Think of a **Physical Stack of Books**. If you place Book A, then Book B, then Book C on a table, Book C is on top. To get Book A, you must first remove C, then remove B. You cannot pull A out from the bottom without the whole pile collapsing!

2. Core Operations

Unlike vectors or lists, a stack only supports a very limited set of actions. This simplicity is exactly what makes it so powerful for specific computational tasks.

1. Push

Adds an element to the top of the stack. The stack size increases by 1.

2. Pop

Removes the topmost element. The stack size decreases by 1. **Critical Note:** In C++ STL, `pop()` is a void function; it does not return the value it removes.

3. Top

Returns a reference to the top element. This allows you to see the data without deleting it (often called "Peeking").

4. Empty & Size

Check if the stack is containing anything and how many items are currently held.

3. Implementing the Stack

To use the stack, you must include the `<stack>` header. It is part of the `std` namespace.

```
#include <iostream>
#include <stack>

int main() {
    // Declaration: stack<Type> name;
    std::stack<int> s;

    // Adding data
    s.push(10);
    s.push(20);
    s.push(30);

    // View the top
    std::cout << "Top is: " << s.top(); // Output: 30

    // Remove the top
    s.pop(); // 30 is gone

    std::cout << "New top: " << s.top(); // Output: 20

    return 0;
}
```

4. The Danger: Underflow

One of the most common bugs when using a stack is trying to access the `top()` or calling `pop()` on an **empty stack**. This results in undefined behavior and usually a program crash.

Safety Golden Rule: Always check `s.empty()` before you ever use `s.top()` or `s.pop()`.

```
if (!s.empty()) {
    std::cout << s.top();
    s.pop();
} else {
    std::cout << "Error: Stack is empty!";
}
```

5. Iteration (Emptying the Stack)

Because stacks don't have iterators (no `begin()` or `end()`), you cannot use a for-loop or a range-based loop to print a stack. To see every element, you must **destroy** the stack as you go.

```
// To print all elements, we "process" them one by one
while (!s.empty()) {
    std::cout << s.top() << " ";
    s.pop(); // This removes the element forever
}
```

If you need to keep the data while printing, you would have to copy the stack into another temporary stack, print the copy, and then discard it.

6. Application: Undo/Redo Logic

Have you ever wondered how 'Ctrl+Z' works in Microsoft Word or Photoshop? It is powered by a stack.

1. Every time you type a word, that action is push () ed onto the Undo-Stack.
2. When you hit Undo, the computer pop () s the last action from the stack and reverses it.
3. That reversed action can then be push () ed onto a "Redo-Stack".

This is only possible because we always undo the **most recent** thing first—classic LIFO logic.

7. Application: Matching Brackets

Compilers use stacks to check if your code's brackets are balanced. If you write `( [ ) ]`, the compiler knows it's wrong using this algorithm:

- If you see an opening bracket `( [ {`, push it onto the stack.
- If you see a closing bracket `) ] }`, check the top of the stack.
- If the top matches (e.g., `)` matches `(`), pop it.
- If the stack is empty at the end, your code is valid!

8. The Call Stack & Recursion

Even if you don't explicitly use `std::stack`, your computer is using one right now. The **Call Stack** stores information about active functions.

When Function A calls Function B:

1. Function A's current state is push () ed onto the call stack.
2. Function B starts running.
3. When Function B finishes, Function A is pop () ed back to the top and continues where it left off.

This is why high-level recursion (functions calling themselves too many times) leads to the famous "**Stack Overflow**" error.

9. Stack vs. Vector

While a vector can do everything a stack can, the stack container adapter provides **intent** and **safety**.

Feature	Stack	Vector
Access	Top Only	Any Index
Purpose	Strict LIFO logic	General storage
Iterators	No	Yes

10. Performance: Big O Complexity

The stack is one of the most efficient data structures in existence. Because it only ever deals with the "top," its operations are extremely predictable.

- **Push:** $O(1)$ - Constant time
- **Pop:** $O(1)$ - Constant time
- **Top:** $O(1)$ - Constant time
- **Search:** N/A - Not supported!

If you need to search for an item, a stack is the wrong choice. You should use a **Vector** or **Set** instead.

11. Infix to Postfix Conversion

Computers don't calculate $5 + 2 * 3$ like humans. They use stacks to convert this to "Postfix" notation: $5\ 2\ 3\ *\ +$.

The stack holds the **operators** (+, *, -) and pops them out based on their priority. This is the foundation of how calculators and programming languages evaluate math.

12. Backtracking Algorithms

Stacks are essential for **Backtracking**. Imagine a robot in a maze:

1. Every time the robot makes a turn, it pushes that turn onto a stack.
2. If the robot hits a dead end, it pops the last turn to move backward to the last intersection.
3. It continues this until it finds the exit.

13. Advanced: The Underlying Container

The `std::stack` is actually a "Container Adapter." By default, it uses a `std::deque` to hold its data. However, you can actually tell it to use a `std::vector` or `std::list` instead!

```
// A stack that uses a vector internally
std::stack<int, std::vector<int>> s;
```

99% of programmers just use the default, but it's good to know you have the power to change its internals for specific memory requirements.

14. Summary & Best Practices

Mastering the stack is a rite of passage for every C++ programmer. Remember these final thoughts:

- **LIFO:** Last In, First Out.
- **Encapsulation:** Use a stack when you want to prevent illegal access to middle elements.
- **Efficiency:** Constant time $O(1)$ for all allowed operations.
- **Interviews:** Stacks are very common in technical interviews for string manipulations and DFS (Depth First Search) algorithms.

Stack High. Code Deep.